Level 8: Real Interview Problems (Questions 181–200)

These twenty questions are styled after real "company round" SQL interview questions. Each one states a business problem first, then the query that solves it, then a short breakdown of how the query actually works.

Q181: Amazon orders analysis

PROBLEM

Find products ordered in the last 30 days but with zero sales in the previous 90 days

```
WITH recent_sales AS (
    SELECT DISTINCT product_id
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_date >= CURDATE() - INTERVAL 30 DAY
),
past_sales AS (
    SELECT DISTINCT product_id
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_date BETWEEN CURDATE() - INTERVAL 120 DAY AND CURDATE() - INTERVAL 31 DAY
)
SELECT p.product_name, p.price
FROM products p
WHERE p.product_id IN (SELECT product_id FROM recent_sales)
AND p.product_id NOT IN (SELECT product_id FROM past_sales);
```

HOW THIS WORKS

Two CTEs build two separate ID lists – products sold in the last 30 days, and products sold in the 90-day window before that – and the final query keeps only IDs that appear in the first list but not the second. This "IN one set, NOT IN another set" shape is the standard way to spot products that suddenly came back to life after a quiet spell, without needing a join at all.

Q182: Flipkart sales analysis

PROBLEM

Top 3 sellers by month

```
WITH monthly_sales AS (
    SELECT
        e.name AS seller,
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.amount) AS total_sales
    FROM employees e
    JOIN orders o ON e.emp_id = o.customer_id
    GROUP BY e.name, DATE_FORMAT(o.order_date, '%Y-%m')
),
ranked_sales AS (
    SELECT *,
           RANK() OVER (PARTITION BY month ORDER BY total_sales DESC) AS rn
    FROM monthly_sales
)
SELECT month, seller, total_sales
FROM ranked_sales
WHERE rn <= 3
ORDER BY month DESC, rn;
```

HOW THIS WORKS

This is the “top-N per group” pattern from Level 6, just wrapped in CTEs for readability: the first CTE rolls sales up by seller and month, the second ranks sellers within each month with RANK() OVER (PARTITION BY month …), and the outer query keeps only rn <= 3. Splitting it into two named steps makes it obvious that ranking happens after aggregation, not before.

Q183: Swiggy delivery analysis

PROBLEM

Average delivery time by city and delivery partner

```
CREATE TABLE deliveries (
    delivery_id INT,
    restaurant_id INT,
    delivery_partner_id INT,
    order_time DATETIME,
    delivery_time DATETIME,
    city VARCHAR(50)
);

SELECT
    city,
    delivery_partner_id,
    AVG(TIMESTAMPDIFF(MINUTE, order_time, delivery_time)) AS avg_delivery_time,
    COUNT(*) AS total_deliveries
FROM deliveries
WHERE delivery_time IS NOT NULL
GROUP BY city, delivery_partner_id
HAVING total_deliveries >= 10
ORDER BY city, avg_delivery_time;
```

HOW THIS WORKS

A plain GROUP BY – no CTE needed here. TIMESTAMPDIFF(MINUTE, order_time, delivery_time) turns two timestamps into a single “minutes taken” number per row, AVG() rolls that up per city/partner pair, and HAVING total_deliveries >= 10 filters out partners with too few deliveries to trust the average. Filtering delivery_time IS NOT NULL first excludes orders that haven’t been delivered yet.

Q184: Zomato customer segmentation

PROBLEM

Segment customers based on order frequency and amount

```
WITH customer_metrics AS (
    SELECT
        c.customer_id,
        c.name,
        COUNT(o.order_id) AS order_count,
        SUM(o.amount) AS total_spent,
        AVG(o.amount) AS avg_order_value,
        DATEDIFF(CURDATE(), MAX(o.order_date)) AS days_since_last_order
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.name
)
SELECT
    name,
    order_count,
    total_spent,
    CASE
        WHEN order_count >= 10 AND total_spent > 100000 THEN 'Platinum'
        WHEN order_count >= 5 AND total_spent > 50000 THEN 'Gold'
        WHEN order_count >= 2 AND total_spent > 10000 THEN 'Silver'
        WHEN days_since_last_order > 90 THEN 'Churn Risk'
        ELSE 'Bronze'
    END AS customer_segment,
    CASE
        WHEN days_since_last_order < 7 THEN 'Very Active'
        WHEN days_since_last_order < 30 THEN 'Active'
        WHEN days_since_last_order < 90 THEN 'Dormant'
        ELSE 'Inactive'
    END AS activity_status
FROM customer_metrics
ORDER BY total_spent DESC;
```

HOW THIS WORKS

This is a classic RFM-style segmentation: one CTE computes each customer's order count, total spend, and recency (days_since_last_order) in a single pass, then two separate CASE expressions turn those raw numbers into human-readable labels. Notice the CASE branches are checked top-to-bottom – a customer is only "Gold" if they fail the "Platinum" test first – so branch order matters as much as the conditions themselves.

Q185: Uber trip analysis

PROBLEM

Find drivers with highest acceptance rate and earnings

```
CREATE TABLE trips (
    trip_id INT,
    driver_id INT,
    rider_id INT,
    request_time DATETIME,
    pickup_time DATETIME,
    dropoff_time DATETIME,
    status VARCHAR(20),
    fare DECIMAL(10,2),
    distance_km DECIMAL(5,2)
);

WITH driver_metrics AS (
    SELECT
        driver_id,
        COUNT(*) AS total_requests,
        SUM(CASE WHEN status = 'Completed' THEN 1 ELSE 0 END) AS completed_trips,
        SUM(CASE WHEN status = 'Completed' THEN fare ELSE 0 END) AS total_earnings,
        AVG(CASE WHEN status = 'Completed' THEN
            TIMESTAMPDIFF(MINUTE, pickup_time, dropoff_time)
        END) AS avg_trip_time
    FROM trips
    GROUP BY driver_id
)
SELECT
    driver_id,
    completed_trips,
    total_earnings,
    ROUND(completed_trips * 100.0 / total_requests, 2) AS acceptance_rate,
    RANK() OVER (ORDER BY total_earnings DESC) AS earnings_rank
FROM driver_metrics
WHERE total_requests > 50
ORDER BY acceptance_rate DESC, total_earnings DESC;
```

HOW THIS WORKS

SUM(CASE WHEN status = ‘Completed’ THEN 1 ELSE 0 END) is the standard trick for “counting only rows that match a condition” inside a single GROUP BY, used twice here – once to count completed trips, once to total their fares. total_requests (every row, completed or not) stays as a plain COUNT(*) so the acceptance-rate division afterward compares completed trips against all requests, not just completed ones.

Q186: Ola ride cancellation analysis

PROBLEM

Find reasons for high cancellation rate by time of day

```
WITH trip_cancellations AS (
    SELECT
        CASE
            WHEN HOUR(request_time) BETWEEN 6 AND 11 THEN 'Morning'
            WHEN HOUR(request_time) BETWEEN 12 AND 17 THEN 'Afternoon'
            WHEN HOUR(request_time) BETWEEN 18 AND 23 THEN 'Evening'
            ELSE 'Night'
        END AS time_of_day,
        status,
        COUNT(*) AS trip_count
    FROM trips
    WHERE status = 'Cancelled'
    GROUP BY time_of_day, status
),
total_by_time AS (
    SELECT
        CASE
            WHEN HOUR(request_time) BETWEEN 6 AND 11 THEN 'Morning'
            WHEN HOUR(request_time) BETWEEN 12 AND 17 THEN 'Afternoon'
            WHEN HOUR(request_time) BETWEEN 18 AND 23 THEN 'Evening'
            ELSE 'Night'
        END AS time_of_day,
        COUNT(*) AS total_trips
    FROM trips
    GROUP BY time_of_day
)
SELECT
    tc.time_of_day,
    tc.trip_count AS cancellations,
    tt.total_trips,
    ROUND(tc.trip_count * 100.0 / tt.total_trips, 2) AS cancellation_rate
FROM trip_cancellations tc
JOIN total_by_time tt ON tc.time_of_day = tt.time_of_day
ORDER BY cancellation_rate DESC;
```

HOW THIS WORKS

The same CASE expression buckets HOUR(request_time) into four labels in both CTEs – one counts only cancelled trips per bucket, the other counts all trips per bucket – and the join lines the two counts up so the rate can be divided out. Whenever you see the identical CASE logic repeated across two CTEs like this, it's because one side needs a filtered count and the other needs the unfiltered total.

Q187: Netflix user watch patterns

PROBLEM

Find binge-watching sessions (3+ episodes in 24 hours)

```
CREATE TABLE viewing_history (
    user_id INT,
    episode_id INT,
    series_id INT,
    watch_date DATETIME,
    duration INT
);

WITH binge_patterns AS (
    SELECT
        user_id,
        series_id,
        watch_date,
        LAG(watch_date) OVER (PARTITION BY user_id, series_id ORDER BY watch_date) AS prev_watch,
        COUNT(*) OVER (PARTITION BY user_id, series_id
                       ORDER BY watch_date
                       RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW) AS episodes_in_24h
    FROM viewing_history
)
SELECT
    user_id,
    series_id,
    MIN(watch_date) AS binge_start,
    MAX(watch_date) AS binge_end,
    episodes_in_24h
FROM binge_patterns
WHERE episodes_in_24h >= 3
GROUP BY user_id, series_id, episodes_in_24h
ORDER BY episodes_in_24h DESC;
```

HOW THIS WORKS

The interesting part is the window frame: RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW is a time-based window rather than a row-count-based one – instead of “the previous 2 rows” it means “every row whose watch_date falls within the last 24 hours of this row’s watch_date”. COUNT(*) over that frame gives a rolling episode count per user/series, and keeping only episodes_in_24h >= 3 surfaces genuine binge sessions. (LAG here is computed but unused in the final SELECT – in practice you’d drop it or use it to also report the gap between episodes.)

Q188: Spotify song popularity analysis

PROBLEM

Find songs that increased in popularity month-over-month

```
CREATE TABLE song_plays (
    song_id INT,
    play_date DATE,
    plays_count INT,
    unique_listeners INT
);

WITH monthly_plays AS (
    SELECT
        song_id,
        DATE_FORMAT(play_date, '%Y-%m') AS month,
        SUM(plays_count) AS total_plays,
        SUM(unique_listeners) AS total_listeners
    FROM song_plays
    GROUP BY song_id, DATE_FORMAT(play_date, '%Y-%m')
),
play_growth AS (
    SELECT
        song_id,
        month,
        total_plays,
        LAG(total_plays) OVER (PARTITION BY song_id ORDER BY month) AS prev_month_plays,
        LAG(total_listeners) OVER (PARTITION BY song_id ORDER BY month) AS prev_month_listeners
    FROM monthly_plays
)
SELECT
    song_id,
    month,
    total_plays,
    prev_month_plays,
    ROUND((total_plays - prev_month_plays) * 100.0 / prev_month_plays, 2) AS growth_percent
FROM play_growth
WHERE prev_month_plays IS NOT NULL
AND total_plays > prev_month_plays
ORDER BY growth_percent DESC
LIMIT 10;
```

HOW THIS WORKS

LAG(total_plays) OVER (PARTITION BY song_id ORDER BY month) fetches “last month’s total_plays for this same song” onto the current row, without any self-join. Once that value sits alongside this month’s number, a plain percentage-change formula does the rest. prev_month_plays IS NOT NULL drops each song’s first month, which has no prior month to compare against.

Q189: YouTube video performance

PROBLEM

Identify videos with high engagement ratio (likes + comments / views)

```
CREATE TABLE videos (
    video_id INT,
    upload_date DATE,
    views INT,
    likes INT,
    comments INT,
    shares INT,
    category VARCHAR(50)
);

SELECT
    video_id,
    category,
    views,
    likes,
    comments,
    shares,
    ROUND((likes + comments + shares) * 100.0 / views, 2) AS engagement_rate,
    CASE
        WHEN views > 1000000 THEN 'Viral'
        WHEN views > 100000 THEN 'Popular'
        WHEN views > 10000 THEN 'Rising'
        ELSE 'New'
    END AS video_tier
FROM videos
WHERE views > 1000
ORDER BY engagement_rate DESC
LIMIT 20;
```

HOW THIS WORKS

No CTE or window function needed – just row-level arithmetic. engagement_rate is computed directly per row as (likes + comments + shares) / views, and video_tier buckets the raw view count with a CASE expression. Multiplying by 100.0 (not 100) forces the division into floating-point math before ROUND(), which is what keeps the result from truncating to a whole number.

Q190: Instagram follower growth

PROBLEM

Calculate follower growth rate by month

```
CREATE TABLE follower_history (
    account_id INT,
    date DATE,
    followers INT,
    following INT,
    posts_count INT
);

WITH monthly_growth AS (
    SELECT
        account_id,
        DATE_FORMAT(date, '%Y-%m') AS month,
        MIN(followers) AS start_followers,
        MAX(followers) AS end_followers,
        MIN(posts_count) AS start_posts,
        MAX(posts_count) AS end_posts
    FROM follower_history
    GROUP BY account_id, DATE_FORMAT(date, '%Y-%m')
)
SELECT
    account_id,
    month,
    start_followers,
    end_followers,
    end_followers - start_followers AS net_gain,
    ROUND((end_followers - start_followers) * 100.0 / start_followers, 2) AS growth_rate,
    (end_posts - start_posts) AS new_posts
FROM monthly_growth
WHERE start_followers > 0
ORDER BY growth_rate DESC;
```

HOW THIS WORKS

Rather than LAG-ing a previous row, this squeezes “start of month” and “end of month” out of a single GROUP BY: since follower_history logs one row per day, MIN(followers) within a month is (assuming followers only grow) the earliest reading and MAX(followers) the latest. It’s a lighter-weight alternative to LAG() that works whenever a metric only moves in one direction across the days being grouped.

Q191: Banking transaction analysis

PROBLEM

Identify suspicious transactions (> 3x average, > 5 transactions/day)

```
CREATE TABLE bank_transactions (
    transaction_id INT,
    customer_id INT,
    transaction_date DATETIME,
    amount DECIMAL(10,2),
    type VARCHAR(20),
    status VARCHAR(20)
);

WITH customer_avg AS (
    SELECT
        customer_id,
        AVG(amount) AS avg_transaction,
        STDDEV(amount) AS stddev_transaction
    FROM bank_transactions
    WHERE status = 'Completed'
    GROUP BY customer_id
),
daily_counts AS (
    SELECT
        customer_id,
        DATE(transaction_date) AS day,
        COUNT(*) AS daily_count
    FROM bank_transactions
    GROUP BY customer_id, DATE(transaction_date)
)
SELECT
    t.transaction_id,
    t.customer_id,
    t.amount,
    c.avg_transaction,
    t.amount / c.avg_transaction AS multiplier,
    CASE
        WHEN t.amount > c.avg_transaction + 3 * c.stddev_transaction THEN 'Extreme'
        WHEN t.amount > c.avg_transaction * 2 THEN 'High'
        ELSE 'Normal'
    END AS risk_level
FROM bank_transactions t
JOIN customer_avg c ON t.customer_id = c.customer_id
WHERE t.amount > c.avg_transaction * 2
OR (t.customer_id, DATE(t.transaction_date)) IN (
    SELECT customer_id, day FROM daily_counts WHERE daily_count > 5
)
ORDER BY t.amount DESC;
```

HOW THIS WORKS

Two independent CTEs each answer one half of “what does normal look like for this customer” – customer_avg gives a per-customer baseline (mean and standard deviation), daily_counts gives a per-day transaction frequency. The WHERE clause then flags a transaction if either condition trips: its amount is more than double that customer’s average, OR it belongs to a (customer, day) pair with more than 5 transactions. The tuple form (t.customer_id, DATE(t.transaction_date)) IN (...) is a compact way to check two columns against a subquery at once, instead of writing a second join.

Q192: ATM withdrawal patterns

PROBLEM

Find ATMs with most withdrawals and average amount by time

```
CREATE TABLE atm_transactions (
    atm_id INT,
    transaction_time DATETIME,
    amount DECIMAL(10,2),
    customer_id INT,
    status VARCHAR(20)
);

SELECT
    atm_id,
    DATE(transaction_time) AS day,
    HOUR(transaction_time) AS hour,
    COUNT(*) AS withdrawals,
    AVG(amount) AS avg_amount,
    SUM(amount) AS total_amount,
    RANK() OVER (PARTITION BY DATE(transaction_time) ORDER BY COUNT(*) DESC) AS withdrawal_rank
FROM atm_transactions
WHERE status = 'Successful'
AND transaction_time >= CURDATE() - INTERVAL 30 DAY
GROUP BY atm_id, DATE(transaction_time), HOUR(transaction_time)
HAVING withdrawals >= 10
ORDER BY day DESC, withdrawal_rank;
```

HOW THIS WORKS

This mixes an aggregate window function directly into a GROUP BY query: COUNT*() OVER (PARTITION BY DATE(…) …) is allowed to reference the same COUNT()* that GROUP BY just computed, because window functions always run after grouping. The result is each atm/day/hour bucket ranked against every other bucket from that same day, so you can see which ATM was busiest on any given day without a separate query.

Q193: Hospital patient readmission analysis

PROBLEM

Calculate 30-day readmission rate by department

```
CREATE TABLE admissions (
    admission_id INT,
    patient_id INT,
    admission_date DATE,
    discharge_date DATE,
    department VARCHAR(50),
    diagnosis VARCHAR(100)
);

WITH readmissions AS (
    SELECT
        a1.patient_id,
        a1.department,
        a1.admission_date,
        a1.discharge_date,
        a2.admission_date AS readmission_date,
        DATEDIFF(a2.admission_date, a1.discharge_date) AS days_between
    FROM admissions a1
    LEFT JOIN admissions a2 ON a1.patient_id = a2.patient_id
        AND a2.admission_date > a1.admission_date
        AND DATEDIFF(a2.admission_date, a1.discharge_date) <= 30
),
department_stats AS (
    SELECT
        department,
        COUNT(DISTINCT patient_id) AS unique_patients,
        COUNT(DISTINCT CASE WHEN readmission_date IS NOT NULL THEN patient_id END) AS readmitted_patients
    FROM readmissions
    GROUP BY department
)
SELECT
    department,
    unique_patients,
    readmitted_patients,
    ROUND(readmitted_patients * 100.0 / unique_patients, 2) AS readmission_rate
FROM department_stats
WHERE unique_patients >= 10
ORDER BY readmission_rate DESC;
```

HOW THIS WORKS

The readmissions CTE is a self-join on the same admissions table (a1 = the original stay, a2 = any later stay by the same patient within 30 days of discharge) – structurally similar to the self-referencing joins used for hierarchies earlier, but joined by patient_id and a date condition instead of a manager_id. LEFT JOIN keeps patients with no readmission too (their readmission_date is NULL), which is exactly what COUNT(DISTINCT CASE WHEN readmission_date IS NOT NULL THEN patient_id END) needs in order to count only the patients who really did come back.

Q194: Student marks analysis

PROBLEM

Find top 3 students in each subject with average > 80

```
CREATE TABLE marks (
    student_id INT,
    subject VARCHAR(50),
    marks INT,
    exam_date DATE
);

WITH student_subject_scores AS (
    SELECT
        student_id,
        subject,
        AVG(marks) AS avg_marks,
        COUNT(*) AS exams_taken
    FROM marks
    GROUP BY student_id, subject
    HAVING avg_marks > 80
),
ranked_students AS (
    SELECT
        student_id,
        subject,
        avg_marks,
        RANK() OVER (PARTITION BY subject ORDER BY avg_marks DESC) AS subject_rank
    FROM student_subject_scores
)
SELECT
    student_id,
    subject,
    ROUND(avg_marks, 2) AS avg_score,
    subject_rank
FROM ranked_students
WHERE subject_rank <= 3
ORDER BY subject, subject_rank;
```

HOW THIS WORKS

Two filters happen at two different stages, which is why this needs two CTEs rather than one: HAVING avg_marks > 80 filters out weak students before ranking even starts, then RANK() OVER (PARTITION BY subject ...) ranks only the students who survived that filter, and subject_rank <= 3 keeps the podium. Doing the HAVING filter and the rank filter in the same WHERE clause wouldn't work, since a WHERE clause can't reference a window function's own output.

Q195: E-commerce revenue analysis by product category

PROBLEM

Find categories with highest growth in revenue

```
WITH monthly_category_revenue AS (
    SELECT
        p.category,
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(oi.quantity * oi.price) AS revenue
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.status = 'Completed'
    GROUP BY p.category, DATE_FORMAT(o.order_date, '%Y-%m')
),
category_growth AS (
    SELECT
        category,
        month,
        revenue,
        LAG(revenue) OVER (PARTITION BY category ORDER BY month) AS prev_month_revenue
    FROM monthly_category_revenue
)
SELECT
    category,
    month,
    revenue,
    prev_month_revenue,
    ROUND((revenue - prev_month_revenue) * 100.0 / prev_month_revenue, 2) AS growth_percent,
    CASE
        WHEN revenue > prev_month_revenue THEN 'Growing'
        WHEN revenue < prev_month_revenue THEN 'Declining'
        ELSE 'Stable'
    END AS trend
FROM category_growth
WHERE prev_month_revenue IS NOT NULL
ORDER BY growth_percent DESC;
```

HOW THIS WORKS

Same LAG-based month-over-month shape as Q188's Spotify question and Q190's Instagram question – by this point in the level the pattern should be recognizable on sight: aggregate to one row per group-per-month, LAG() that revenue within each group, then compare current vs. previous. The only new piece here is the three-way join (products, order_items, orders) needed just to get category and revenue onto the same row in the first place.

Q196: Warehouse inventory optimization

PROBLEM

Identify products with low turnover (aged inventory)

```
CREATE TABLE inventory_items (
    item_id INT,
    product_id INT,
    warehouse_id INT,
    quantity INT,
    last_restock_date DATE,
    sales_last_30_days INT
);

WITH inventory_metrics AS (
    SELECT
        product_id,
        warehouse_id,
        quantity,
        DATEDIFF(CURDATE(), last_restock_date) AS days_since_restock,
        sales_last_30_days,
        CASE
            WHEN sales_last_30_days = 0 AND DATEDIFF(CURDATE(), last_restock_date) > 90 THEN 'Critical'
            WHEN sales_last_30_days < 5 AND DATEDIFF(CURDATE(), last_restock_date) > 60 THEN 'Excess'
            WHEN quantity < 10 AND sales_last_30_days > 0 THEN 'Low Stock'
            ELSE 'Normal'
        END AS inventory_status
    FROM inventory_items
)
SELECT
    product_id,
    warehouse_id,
    quantity,
    days_since_restock,
    sales_last_30_days,
    inventory_status,
    ROUND(quantity * 1.0 / NULLIF(sales_last_30_days, 0), 2) AS days_of_inventory
FROM inventory_metrics
WHERE inventory_status != 'Normal'
ORDER BY FIELD(inventory_status, 'Critical', 'Low Stock', 'Excess'), days_since_restock DESC;
```

HOW THIS WORKS

NULLIF(sales_last_30_days, 0) swaps a 0 for NULL right before the division, which is what stops quantity / 0 from throwing a divide-by-zero error – dividing by NULL just quietly produces NULL instead. FIELD(inventory_status, ‘Critical’, ‘Low Stock’, ‘Excess’) is a handy way to sort text values into a custom order (not alphabetical) by giving each string a position number to sort by, so the most urgent status prints first.

Q197: Flight booking analysis

PROBLEM

Find optimal pricing by analyzing booking patterns

```
CREATE TABLE flight_bookings (
    booking_id INT,
    flight_id INT,
    booking_date DATE,
    travel_date DATE,
```

```
    price DECIMAL(10,2),
    seats_booked INT,
    customer_id INT
);

WITH booking_metrics AS (
    SELECT
        flight_id,
        booking_date,
        travel_date,
        price,
        seats_booked,
        DATEDIFF(travel_date, booking_date) AS days_in_advance,
        CASE
            WHEN DATEDIFF(travel_date, booking_date) > 60 THEN 'Very Early'
            WHEN DATEDIFF(travel_date, booking_date) > 30 THEN 'Early'
            WHEN DATEDIFF(travel_date, booking_date) > 14 THEN 'Mid'
            WHEN DATEDIFF(travel_date, booking_date) > 7 THEN 'Late'
            ELSE 'Last Minute'
        END AS booking_window
    FROM flight_bookings
),
price_analysis AS (
    SELECT
        flight_id,
        booking_window,
        AVG(price) AS avg_price,
        SUM(seats_booked) AS total_seats,
        COUNT(*) AS bookings_count
    FROM booking_metrics
    GROUP BY flight_id, booking_window
)
SELECT
    flight_id,
    booking_window,
    ROUND(avg_price, 2) AS avg_price,
    total_seats,
    bookings_count,
    ROUND(AVG(avg_price) OVER (PARTITION BY flight_id), 2) AS flight_avg_price,
    ROUND((avg_price - AVG(avg_price) OVER (PARTITION BY flight_id)) /
          AVG(avg_price) OVER (PARTITION BY flight_id) * 100, 2) AS price_variance
FROM price_analysis
ORDER BY flight_id, FIELD(booking_window, 'Very Early', 'Early', 'Mid', 'Late', 'Last Minute');
```

HOW THIS WORKS

Three layers stack here: booking_metrics buckets each individual booking into a “how far in advance” window, price_analysis aggregates those into one row per flight/window, and the outer SELECT adds AVG(avg_price) OVER (PARTITION BY flight_id) – a window function computed on top of already-aggregated data – to get each flight’s overall average price for comparison. price_variance then shows, as a percentage, how much pricier or cheaper each booking window is versus that flight’s own average, which is exactly the “how much does booking early actually save” answer the question is really asking for.

Q198: Hotel reservation analysis

PROBLEM

Find peak occupancy periods and predict demand

```
CREATE TABLE reservations (
    reservation_id INT,
    hotel_id INT,
    check_in_date DATE,
    check_out_date DATE,
    rooms_booked INT,
    total_guests INT,
    total_amount DECIMAL(10,2),
    status VARCHAR(20)
);

WITH daily_occupancy AS (
    SELECT hotel_id, check_in_date AS date, rooms_booked, 'Check-in' AS event_type
    FROM reservations WHERE status = 'Confirmed'
    UNION ALL
    SELECT hotel_id, check_out_date AS date, -rooms_booked, 'Check-out' AS event_type
    FROM reservations WHERE status = 'Confirmed'
),
occupancy_by_date AS (
    SELECT
        hotel_id,
        date,
        SUM(rooms_booked) AS net_rooms_change,
        SUM(SUM(rooms_booked)) OVER (PARTITION BY hotel_id ORDER BY date) AS occupied_rooms
    FROM daily_occupancy
    GROUP BY hotel_id, date
)
SELECT
    hotel_id, date, occupied_rooms,
    LAG(occupied_rooms) OVER (PARTITION BY hotel_id ORDER BY date) AS prev_day_occupancy,
    occupied_rooms - LAG(occupied_rooms) OVER (PARTITION BY hotel_id ORDER BY date) AS daily_change,
    CASE
        WHEN occupied_rooms > (SELECT AVG(occupied_rooms) FROM occupancy_by_date WHERE hotel_id = o.hotel_id) * 1.2
        THEN 'Peak'
        WHEN occupied_rooms < (SELECT AVG(occupied_rooms) FROM occupancy_by_date WHERE hotel_id = o.hotel_id) * 0.8
        THEN 'Low'
        ELSE 'Normal'
    END AS occupancy_tier
FROM occupancy_by_date o
WHERE date >= CURDATE()
ORDER BY hotel_id, date;
```

HOW THIS WORKS

The clever setup is daily_occupancy: instead of computing “how many rooms are occupied on date X” directly, it turns every reservation into two signed events – a +rooms_booked row on check-in and a -rooms_booked row on check-out – stacked with UNION ALL. A running total of those signed events (SUM(SUM(...)) OVER (ORDER BY date), summing an already-summed daily total) then naturally produces the occupied-room count for every date, rising on check-ins and falling on check-outs, the same running-balance idea as Q177’s credits/debits but applied to room-nights instead of money.

Q199: Stock market transaction analysis

PROBLEM

Calculate portfolio performance with buy/sell patterns

```
CREATE TABLE stock_trades (
    trade_id INT,
    user_id INT,
    stock_symbol VARCHAR(10),
    trade_date DATE,
    trade_type VARCHAR(4), -- 'BUY', 'SELL'
    shares INT,
    price DECIMAL(10,2)
);

WITH position_calculations AS (
    SELECT
        user_id, stock_symbol, trade_date, trade_type, shares, price,
        CASE WHEN trade_type = 'BUY' THEN shares ELSE -shares END AS net_shares,
        CASE WHEN trade_type = 'BUY' THEN shares * price ELSE -shares * price END AS net_value
    FROM stock_trades
),
daily_positions AS (
    SELECT
        user_id, stock_symbol, trade_date, shares, price, net_shares, net_value,
        SUM(net_shares) OVER (PARTITION BY user_id, stock_symbol ORDER BY trade_date) AS cumulative_shares,
        SUM(net_value) OVER (PARTITION BY user_id, stock_symbol ORDER BY trade_date) AS cost_basis
    FROM position_calculations
)
SELECT
    user_id, stock_symbol, trade_date, trade_type, shares, price, cumulative_shares, cost_basis,
    CASE WHEN cumulative_shares > 0 THEN ROUND(cost_basis / cumulative_shares, 2) ELSE 0 END AS avg_cost_per_share,
    RANK() OVER (PARTITION BY user_id ORDER BY trade_date DESC) AS recent_trade_rank
FROM daily_positions
WHERE trade_type = 'SELL'
ORDER BY user_id, trade_date DESC;
```

HOW THIS WORKS

BUY and SELL rows are first converted into signed net_shares and net_value (a SELL becomes negative), and then two running totals – SUM(net_shares) OVER (...) and SUM(net_value) OVER (...), both ordered by trade_date – turn that trade-by-trade history into a live position size and running cost basis per user/stock, the same signed-running-total idea as Q177 and Q198 applied a third time to a different domain. avg_cost_per_share = cost_basis / cumulative_shares at that point in time is then just one more column computed from the two running totals.

Q200: End-to-End Employee Management System SQL Challenge

```
-- Complete employee management dashboard
WITH employee_details AS (
    SELECT
        e.emp_id, e.name, e.age, e.gender, e.salary, e.department, e.designation,
        e.hire_date, e.experience_years, e.bonus,
        m.name AS manager_name, d.location AS dept_location, d.budget AS dept_budget
    FROM employees e
    LEFT JOIN employees m ON e.manager_id = m.emp_id
    LEFT JOIN departments d ON e.department = d.dept_name
),
department_stats AS (
    SELECT
        department, COUNT(*) AS employee_count, AVG(salary) AS avg_salary,
        SUM(salary) AS total_salary, AVG(experience_years) AS avg_experience,
        MAX(salary) AS max_salary, MIN(salary) AS min_salary
    FROM employee_details
    GROUP BY department
),
project_stats AS (
    SELECT department, COUNT(*) AS active_projects, SUM(budget) AS project_budget
    FROM projects WHERE status = 'Active'
    GROUP BY department
),
order_stats AS (
    SELECT e.department, COUNT(o.order_id) AS total_orders, SUM(o.amount) AS total_revenue,
           AVG(o.amount) AS avg_order_value
    FROM employees e
    LEFT JOIN orders o ON e.emp_id = o.customer_id
    GROUP BY e.department
)
SELECT
    ed.department, ed.employee_count, ROUND(ed.avg_salary, 2) AS avg_salary,
    ROUND(ed.total_salary, 2) AS total_salary, ROUND(ed.avg_experience, 1) AS avg_experience_years,
    COALESCE(ps.active_projects, 0) AS active_projects,
    COALESCE(ROUND(os.total_revenue, 2), 0) AS total_revenue,
    ROUND(COALESCE(os.total_revenue, 0) / NULLIF(ed.total_salary, 0) * 100, 2) AS revenue_to_salary_ratio,
    RANK() OVER (ORDER BY COALESCE(os.total_revenue, 0) DESC) AS revenue_rank
FROM department_stats ed
LEFT JOIN project_stats ps ON ed.department = ps.department
LEFT JOIN order_stats os ON ed.department = os.department
ORDER BY revenue_rank;
```

HOW THIS WORKS

The capstone question, and deliberately the longest query in the book – but every individual piece has already appeared earlier in this level or the ones before it. Four CTEs each aggregate one business area (employee details, department stats, project stats, order stats) completely independently, and only the final SELECT joins all four together with LEFT JOIN, so a department with zero active projects or zero orders still shows up instead of disappearing. COALESCE(…, 0) papers over the NULLs a LEFT JOIN produces for departments with no matching projects/orders, and NULLIF(ed.total_salary, 0) guards the final ratio against a divide-by-zero the same way Q196 did. If you can trace what each CTE contributes on its own before looking at the JOINs, you've fully internalized the pattern this whole level was building toward.